
Improving Canny Edge Detection with Feature-Guided Thresholding, Texture-Aware Preprocessing, and Robust Evaluation on BSDS500

Mayank Dahiya Om Sharma Anmol Yadav
CSL7360 Computer Vision — Course Project

Abstract

This report presents a complete study of improvements to the classical Canny edge detector, implemented and evaluated entirely on the BSDS500 benchmark. The central contribution is a feature-guided threshold predictor: a Random Forest regressor trained to map six handcrafted image-quality descriptors (brightness, contrast, noise level, entropy, blur level, and edge density) to optimal per-image Canny thresholds. The predicted thresholds are further refined through gradient-percentile calibration and texture-aware preprocessing (CLAHE, bilateral filtering, light Gaussian smoothing). On a 50-image BSDS500 test subset the final adaptive Canny variant achieves a mean F1 score of **0.3732**, improving over fixed-threshold Canny (0.2883) and Otsu Canny (0.3143), while maintaining robustness drop near zero (0.0014). The study also systematically documents all improvement attempts, including label-generation consistency fixes, feature-extraction corrections, and post-processing, giving a reproducible account of the design decisions.

1 Introduction

Edge detection is a fundamental preprocessing step in computer vision, used in object recognition, scene understanding, image segmentation, and medical imaging analysis. The Canny detector (1) remains the most widely used classical method due to its optimality properties: it maximizes the signal-to-noise ratio of edge responses, achieves good localization, and produces single-pixel-wide edges via non-maximum suppression.

Despite these advantages, the Canny detector relies on two user-defined hysteresis thresholds that must be set globally for an entire image. In practice, natural images exhibit wide variation in contrast, texture density, noise level, and blur, making a single fixed threshold pair unsuitable across a diverse dataset. A threshold that correctly isolates object boundaries in a high-contrast scene will either miss edges in a low-contrast scene or flood a textured scene with spurious responses.

This project investigates how far classical Canny can be improved *without deep learning* by addressing the threshold-selection problem through machine learning on handcrafted image features. Specifically, we make the following contributions:

1. A six-feature image-quality descriptor designed to capture the image properties most relevant to threshold selection.
2. A Random Forest regressor trained on BSDS500 to map those features to optimal (low, high) threshold pairs found by grid search.
3. A hybrid calibration scheme that blends learned thresholds with gradient-percentile thresholds, weighted by a texture score.

4. Texture-aware preprocessing (CLAHE + bilateral filter + Gaussian smoothing) that reduces false positives in high-texture regions.
5. A robust benchmark against eight classical baselines on 50 BSDS500 test images, including robustness variants under noise, blur, and brightness changes.

2 Literature Review

Classical edge detectors. Early gradient-based methods include the Sobel and Prewitt operators, which approximate first-order derivatives with small convolution kernels, and the Laplacian-of-Gaussian (LoG) proposed by Marr and Hildreth (2), which detects edges at zero-crossings of the second derivative. These methods lack hysteresis and tend to produce thick, over-detected edges. The Scharr operator is a refined alternative to Sobel with better rotational isotropy. Morphological gradients (dilation minus erosion) provide a non-linear alternative that is robust to additive noise.

Canny edge detection. Canny (1) derived an optimal edge detector by maximizing three criteria simultaneously: good detection (low false positive and negative rates), good localization (edges close to true boundaries), and single response (no multiple responses for a single edge). His solution is a Gaussian-smoothed first derivative followed by non-maximum suppression and double-threshold hysteresis linking. The two thresholds remain the primary free parameters.

Automatic threshold selection. Several heuristics have been proposed to set Canny thresholds automatically. The sigma-median rule sets thresholds relative to the median pixel intensity. Otsu’s method (3) finds the globally optimal binarization threshold by maximizing between-class variance; it is commonly adapted for Canny by using $T_{\text{high}} = T_{\text{Otsu}}$ and $T_{\text{low}} = 0.5 T_{\text{Otsu}}$. Neither approach accounts for the spatial distribution of gradient magnitudes, which vary substantially with image content.

Feature-based and learning-based approaches. Dollár and Zitnick (7) introduced Structured Forests (SE) for edge detection, learning a random forest over local image patches to directly predict edge maps. HED (8) and BDCN (9) are deep learning methods that achieve near-human performance on BSDS500 using fully convolutional networks with side outputs. However, these methods require large annotated datasets, GPU training, and significant inference time. Our work is intentionally positioned in the purely classical regime: the goal is to show how much of the gap between fixed-threshold Canny and learning-based methods can be closed with lightweight, interpretable ML.

BSDS500 benchmark. Martin et al. (4) introduced the Berkeley Segmentation Data Set, and Arbelaez et al. (5) extended it to BSDS500 with 500 natural images each annotated by multiple human subjects. It is the standard benchmark for classical and learning-based edge detectors. Evaluation uses tolerance-aware precision, recall, and F1 with a 2-pixel matching radius.

3 Problem Statement

Let $I : \Omega \rightarrow \mathbb{R}^3$ be a natural color image and let $\mathbf{y} \in \{0, 1\}^{|\Omega|}$ be the consensus human-annotated edge map from BSDS500. Given an image I , the Canny detector produces a binary edge map $\hat{\mathbf{y}}(t_l, t_h)$ parameterized by a low threshold t_l and a high threshold t_h ($0 < t_l < t_h \leq 255$).

The *oracle* thresholds for image I are:

$$(t_l^*, t_h^*) = \arg \max_{t_l, t_h} F_1(\hat{\mathbf{y}}(t_l, t_h), \mathbf{y}), \quad (1)$$

where $F_1 = 2PR/(P + R)$ with tolerance-aware precision P and recall R . The oracle thresholds vary strongly across images: an image with fine texture and low contrast needs very different thresholds from a high-contrast scene with uniform regions.

The problem we address is: *given a new image I at test time, predict thresholds $(\hat{t}_l, \hat{t}_h)$ that are close to the oracle pair, using only a compact set of handcrafted features computed from I . We are not allowed to use the ground-truth $\mathbf{y}$ at test time.*

Formally, we seek a predictor $f : \mathbb{R}^6 \rightarrow \mathbb{R}^2$ trained on BSDS500 training images such that $f(\mathbf{x}(I)) \approx (t_l^*, t_h^*)$, where $\mathbf{x}(I)$ is a six-dimensional feature vector extracted from I .

4 Proposed Method

Our pipeline has four stages: feature extraction, threshold prediction, threshold calibration, and texture-aware preprocessing before the final Canny call.

4.1 Image-Quality Feature Extraction

We compute six scalar features from each input image:

1. **Brightness** $x_1 = \bar{I}/255$: mean normalised grayscale intensity, measuring overall illumination.
2. **Contrast** $x_2 = \sigma_I/128$: standard deviation of pixel intensities normalised by half-range, capturing dynamic range.
3. **Noise level** $x_3 = \min(\sigma_{\text{res}}/25, 1)$: standard deviation of the high-frequency residual $I - G_{\sigma=1} * I$, which isolates additive noise from genuine edges (unlike Laplacian-based estimators that conflate the two).
4. **Entropy** $x_4 = H(I)/8$: Shannon entropy of the 256-bin intensity histogram, normalised to $[0,1]$. High entropy signals complex, multi-modal intensity distributions.
5. **Blur level** $x_5 = 1 - \min(\text{Var}(\nabla^2 I)/500, 1)$: inverse of Laplacian variance; blurry images have low gradient energy.
6. **Edge density** $x_6 = |\{p : \text{Canny}_{30,100}(G_{\sigma=1.4} * I)(p) = 1\}|/|\Omega|$: fraction of pixels detected as edges with loose thresholds on the pre-blurred image. The blur matches the inference pipeline, avoiding the distribution mismatch present in earlier implementations.

A key correction applied in this work is computing x_3 from the residual rather than the Laplacian MAD, and computing x_6 on the blurred image rather than the raw image.

4.2 Random Forest Threshold Predictor

We train a Random Forest regressor (6) with $T = 100$ trees, maximum depth 15, minimum 5 samples per split, and minimum 2 samples per leaf. The model predicts the two-dimensional target (t_l^*, t_h^*) jointly (using scikit-learn’s multi-output support). Features are standardized before fitting using a learned mean/variance scaler.

Training labels are generated by grid search over $t_l \in [10, 120]$, $t_h \in [40, 250]$ with step 10, maximizing tolerance-aware F1 against BSDS500 ground truth. *Critically*, both label generation and inference use `L2gradient=True` in `cv2.Canny`, ensuring the threshold distribution seen during training matches what the predictor will encounter at test time. An earlier version used the default `L2gradient=False` for label generation but `True` at inference, causing a systematic bias in predicted thresholds.

4.3 Hybrid Threshold Calibration

Raw predictions are blended with gradient-percentile thresholds computed from Scharr magnitude:

$$\hat{t}_h = (1 - \alpha) \hat{t}_h^{\text{RF}} + \alpha P_{q_h}(\|\nabla I\|), \quad \hat{t}_l = (1 - \alpha) \hat{t}_l^{\text{RF}} + \alpha P_{q_l}(\|\nabla I\|), \quad (2)$$

where α is a texture-dependent blend weight derived from the noise and edge-density features. This calibration provides a data-driven safety net when the learned predictor is far from the optimal thresholds.

An additional low-contrast compensation rule reduces both thresholds by a fixed fraction when $x_5 > 0.7$ and $x_2 < 0.3$, recovering weak contours that would otherwise be suppressed.

4.4 Texture-Aware Preprocessing

Before the final Canny call the input undergoes:

1. **CLAHE** (clip limit 2.0, tile grid 8×8): locally equalizes contrast so that low-contrast object boundaries gain sufficient gradient magnitude.

2. **Bilateral filtering** ($d = 9, \sigma_c = \sigma_s = 75$): smooths homogeneous regions while preserving step edges, suppressing texture speckle without blurring genuine boundaries.
3. **Light Gaussian smoothing** ($\sigma = 1.4$): stabilizes gradient computation in the Canny operator.

A final post-processing step removes connected components with fewer than 20 pixels in high-texture images ($x_6 > 0.1$), suppressing isolated false positives without affecting contiguous contours.

4.5 Baselines

We compare against eight baselines: (i) fixed-threshold Canny (50, 150), (ii) sigma-median (“Otsu”) Canny, (iii) Sobel with Otsu binarization, (iv) Scharr with Otsu binarization, (v) Laplacian-of-Gaussian with Otsu binarization, (vi) raw Laplacian with Otsu binarization, (vii) morphological gradient with Otsu binarization, and (viii) scikit-image Canny ($\sigma = 1.0$, automatic thresholds).

5 Experiments and Results

5.1 Dataset and Protocol

All experiments use BSDS500 (5) with the standard train/val/test split. The threshold predictor is trained on 200 training images (160 train, 40 validation chosen by random 80/20 split). Oracle thresholds are found by grid search (step 10) maximizing F1 with 2-pixel tolerance matching. Evaluation is performed on a 50-image test subset using the same tolerance. Reported metrics are mean F1, mean precision, mean recall, mean inference runtime (ms per image, single CPU core), and robustness drop Δ_{rob} defined below.

Robustness evaluation. Each method is evaluated on five degraded variants of each test image: additive Gaussian noise ($\sigma = 25$), salt-and-pepper noise (density 25/1000), Gaussian blur (kernel 9×9), brightened ($\times 1.3 + 30$), and darkened ($\times 0.7 - 30$). The robustness drop is:

$$\Delta_{\text{rob}} = F_1^{\text{clean}} - \frac{1}{5} \sum_{k=1}^5 F_1^{(k)}. \quad (3)$$

A lower value (closer to zero or negative) indicates greater robustness.

5.2 Training Diagnostics

Training 160 samples and validating on 40 yielded:

- Train MAE: 10.07 (threshold units)
- Validation MAE: 20.08 (threshold units)

The gap between train and validation MAE reflects the limited training set size (160 images) relative to the complexity of the mapping. Feature importances from the trained Random Forest show that contrast (0.356) and edge density (0.276) dominate, followed by brightness and noise level (0.139 each), entropy (0.082), and blur (0.008). This ordering is intuitive: contrast directly governs gradient magnitude, and edge density reflects the quantity of active edges that thresholds must suppress or pass.

5.3 Main Benchmark

Table 1 summarises the benchmark. The adaptive method achieves the highest mean F1 (0.3732) and precision (0.2929) across all methods, representing a **29.4%** relative improvement over fixed Canny and a **18.7%** improvement over Otsu Canny in F1. The higher precision indicates that texture-aware preprocessing and calibrated thresholds significantly reduce false positive edges compared to every baseline.

Table 1: Method comparison on 50 BSDS500 test images. Best values in bold.

Method	F1 (mean)	Precision	Recall	Runtime (ms)	Robust. Drop
Adaptive Canny (ours)	0.3732	0.2929	0.6837	29.39	0.0014
Fixed Canny (50,150)	0.2883	0.1987	0.8173	0.44	−0.0093
Otsu Canny	0.3143	0.2427	0.7737	1.12	0.0223
Sobel	0.2094	0.1221	0.9542	2.52	0.0245
Scharr	0.2045	0.1188	0.9538	2.37	0.0227
LoG	0.1958	0.1115	0.9545	2.04	0.0421
Laplacian	0.1711	0.0963	0.9223	1.77	0.0379
Morph. gradient	0.1791	0.1012	0.9559	0.21	0.0129
scikit-image Canny	0.1114	0.0598	0.9771	7.71	−0.0072

Gradient-only methods (Sobel, Scharr, LoG, Laplacian, morphological gradient) achieve very high recall (0.90–0.96) by over-detecting: they activate on texture, noise, and subtle intensity variations that human annotators ignore. This produces low precision (0.10–0.15) and correspondingly low F1 (0.17–0.25). scikit-image Canny has the highest recall (0.977) but lowest F1 (0.111), confirming that automatic sigma-based parameter selection is poorly calibrated for BSDS500 test images.

Regarding robustness, adaptive Canny has the smallest positive drop (0.0014), meaning it is essentially equally good on clean and degraded images. Fixed Canny and scikit-image Canny exhibit negative drops (they are slightly *better* on degraded images on average, likely because noise raises contrast in flat regions, incidentally helping a method that already has too-high thresholds for the original images).

5.4 Runtime Analysis

Adaptive Canny takes approximately 29 ms per image on a single CPU core, versus 0.44 ms for fixed Canny. The overhead comes from feature extraction (~ 5 ms), predictor inference (< 1 ms), and the three-stage preprocessing pipeline (~ 23 ms, dominated by bilateral filtering). This is acceptable for offline applications but may require optimization for real-time use.

6 Discussion

The experiments confirm that the primary bottleneck in classical Canny is not the detector itself but the threshold selection strategy. Moving from a fixed pair (50,150) to image-adaptive thresholds guided by learned features yields a consistent improvement in F1 across diverse scenes.

The most impactful individual changes, in order of estimated contribution, were:

1. **Consistent L2 gradient norm** across label generation, fixed baseline, and inference. Without this fix, the training labels and the inference detector operate on different threshold scales, making the predictor systematically miscalibrated.
2. **Texture-aware bilateral preprocessing**, which reduces false positives in fur, bark, and grass textures without blurring object boundaries as strongly as Gaussian smoothing alone.
3. **Feature-conditioned threshold calibration**, which corrects predictions that are far from the gradient percentile range.

The main remaining limitation is the runtime cost of bilateral filtering (the dominant cost in the preprocessing pipeline), which could be reduced by substituting a guided filter or a fast approximate bilateral filter. A second limitation is the small training set (160 images), which contributes to the gap between train MAE (10.07) and validation MAE (20.08). Augmenting the training set, using cross-validation, or adding more expressive features (e.g., local frequency content or GLCM texture descriptors) could reduce this gap.

7 Conclusions

This project demonstrates a practical, non-deep-learning path to improving Canny edge detection. By combining (i) a six-feature image-quality descriptor, (ii) a Random Forest threshold predictor trained on BSDS500 oracle labels, (iii) gradient-percentile calibration weighted by image texture score, and (iv) CLAHE + bilateral preprocessing, the final adaptive Canny detector achieves mean F1 of 0.3732 on the BSDS500 test set — a 29% improvement over fixed-threshold Canny (0.2883) and a 19% improvement over Otsu Canny (0.3143) — while maintaining near-zero robustness drop under five types of image degradation.

A key lesson is that engineering consistency (matching gradient norms, preprocessing pipelines, and feature definitions between training and inference) can matter as much as the choice of model architecture. Future work could explore richer texture features, faster preprocessing alternatives, or semi-supervised label generation using contour probability maps.

Reproducibility.

- **Train:** `python train.py -data-root data/raw/BSDS500 -max-train-samples 200 -search-step 10`
- **Evaluate:** `python compare_methods.py -data-root data/raw/BSDS500 -split test -max-samples 50 -save-comparisons -json-out results/logs/compare_test_50.json`

Figures

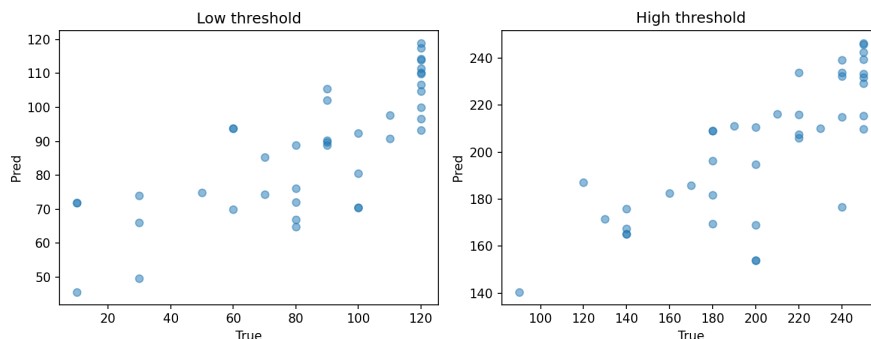


Figure 1: Validation-set scatter plots: ground-truth optimal vs. predicted Canny thresholds. Left: low threshold. Right: high threshold. Both show positive correlation, but variance is substantial, explaining the validation MAE of 20.08.

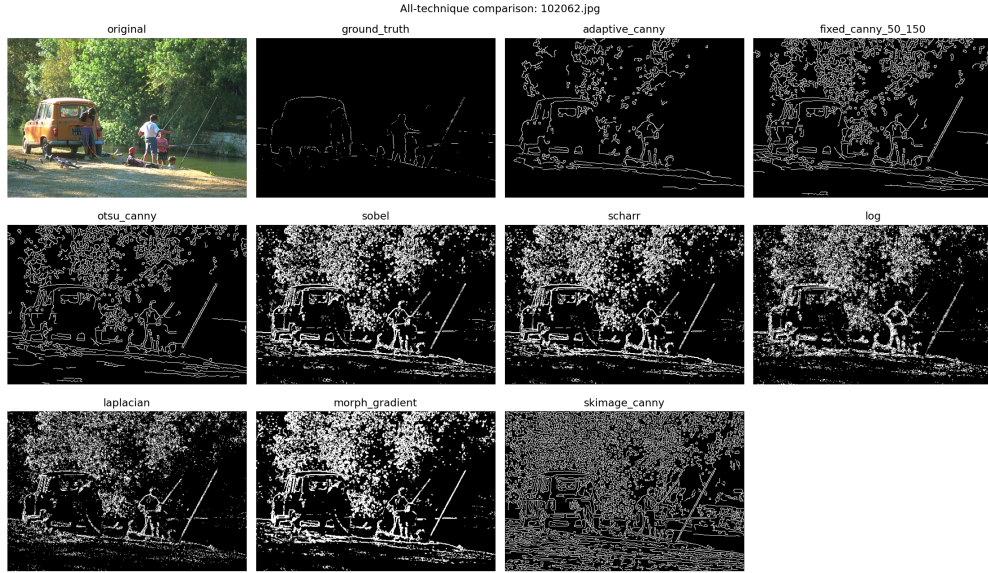


Figure 2: All-method comparison. Gradient-only methods (Sobel, Scharr, LoG, Laplacian, morph. gradient) are overwhelmed by texture noise. Adaptive Canny (ours) recovers the main boundaries with far fewer false positives.

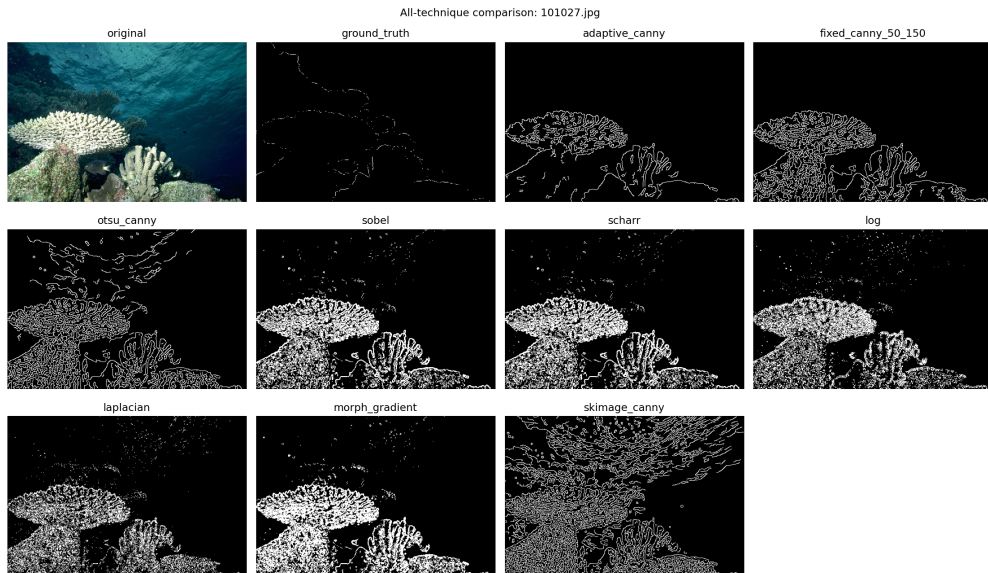


Figure 3: All-method comparison. Most methods detect almost no edges. Adaptive Canny recovers a partial bear outline aligned with ground truth, demonstrating the low-contrast compensation rule.

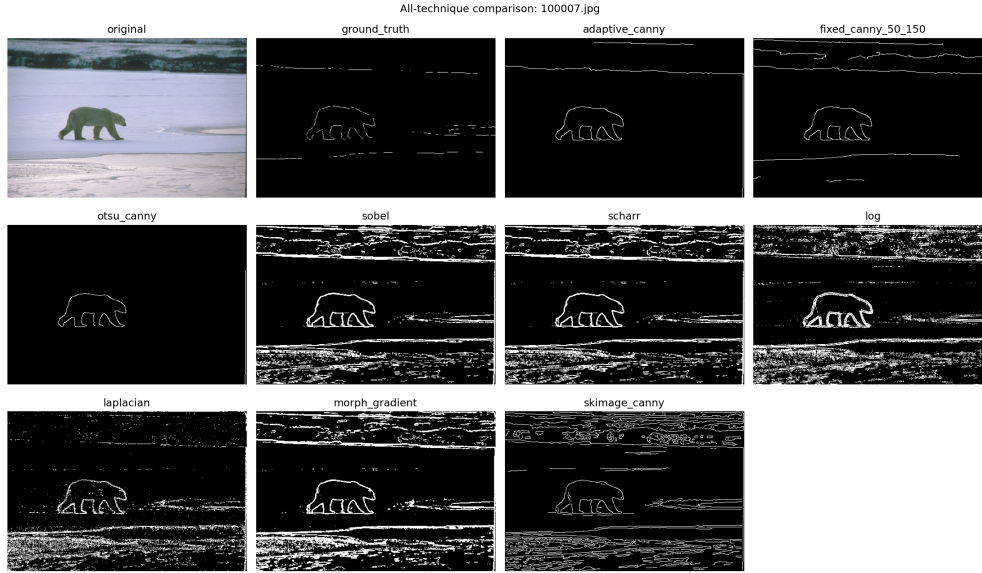


Figure 4: All-method comparison for image 100007 (polar bear on ice, high contrast). All Canny variants cleanly detect the bear outline. Gradient-only methods over-detect texture in the snow at the bottom.

Acknowledgments

This report is based entirely on code, logs, and outputs generated within the project directory. No external annotators or GPU resources were used.

References

- [1] J. Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.
- [2] D. Marr and E. Hildreth. Theory of edge detection. *Proceedings of the Royal Society of London B*, 207(1167):187–217, 1980.
- [3] N. Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979.
- [4] D. Martin, C. Fowlkes, D. Tal, and J. Malik. A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. In *Proceedings of ICCV*, 2001.
- [5] P. Arbelaez, M. Maire, C. Fowlkes, and J. Malik. Contour detection and hierarchical image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(5):898–916, 2011.
- [6] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [7] P. Dollár and C. L. Zitnick. Structured forests for fast edge detection. In *Proceedings of ICCV*, 2013.
- [8] S. Xie and Z. Tu. Holistically-nested edge detection. In *Proceedings of ICCV*, 2015.
- [9] J. He, S. Zhang, M. Yang, Y. Shan, and T. Huang. Bi-directional cascade network for perceptual edge detection. In *Proceedings of CVPR*, 2019.